

SMART TEMPERATURE MONITOR SYSTEM

Introduction to Embedded Systems

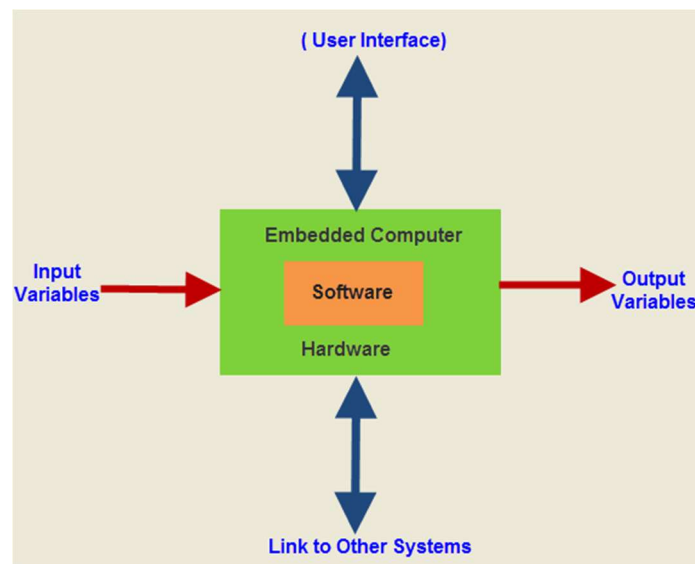
1. What is an Embedded System?

An embedded system is a specialized computer system designed to perform a specific function or a group of functions within a larger system. Unlike general-purpose computers such as laptops or desktops, embedded systems are built for dedicated tasks and usually operate with limited hardware and software resources.

An embedded system consists of both

Hardware: microcontroller/microprocessor, sensors, memory, communication modules, etc.

Software: firmware or embedded programs written in languages like C, C++, or Python.



Basic Structure of an Embedded System

Input Devices: sensors, switches, buttons

Processing Unit: microcontroller or microprocessor

Output Devices: LEDs, motors, displays, buzzers

Memory: stores program and data

Power Supply: provides electrical power

2. Real-World Applications of Embedded Systems

Embedded systems are used in almost every modern electronic device and industry. They play a major role in automation, communication, healthcare, transportation, and consumer electronics.

1. Smart Home Systems
2. Healthcare Devices
3. Robotics and Automation
4. Consumer Electronics

3. Difference between Microprocessor and Microcontroller:

Microprocessor vs Microcontroller		
Memory	Needs external memory & storage, flexible	On-chip memory (RAM, ROM, EEPROM)
Peripheral 	Need additional part via external bus, ideal for complex, powerful applications.	Integrated peripheral SPI, UART, RTI, ADC for direct HW control
Clock Speed 	Fast GHz for heavy duty tasks 	KHz - MHz, efficient for specific tasks. 
Power Consumption 	High power consumption, no low power option. 	Low power consumption with power saving options 
Connectivity	Handles high-speed transfer	supports low to moderate speed communication I2C, SPI, UART
Operating System 	Need full-fledge OS (Linux, Windows)	OS is optional (no-OS or light weight RTOS)
Usecase	For generic computing, high complexity capacity system	For compact systems, battery powered, or logic processing devices
Cost	Higher cost — suited for performance-driven applications	Lower cost — economical for mass production.

2. Component Study

A. Role of Microcontrollers

Microcontrollers are the core processing units used in embedded systems. They receive input from sensors, process the data, and control output devices such as LEDs, motors, displays, and buzzers. They are designed for dedicated real-time applications and are widely used in automation, IoT, robotics, and smart devices.

1. Arduino

Arduino is one of the most popular open-source microcontroller platforms used for learning electronics and embedded systems.

Features:

Easy to program using Arduino IDE

Supports C/C++ programming

Contains digital and analog I/O pins

Low cost and beginner friendly

Large community support

2. ESP32

Espressif Systems developed the ESP32, a powerful microcontroller with built-in Wi-Fi and Bluetooth connectivity.

Features:

Dual-core processor

Built-in Wi-Fi and Bluetooth

High processing speed

Low power consumption

Supports IoT applications

Role in Embedded Systems

3. Raspberry Pi Pico

Raspberry Pi Foundation developed the Raspberry Pi Pico using the RP2040 microcontroller chip.

Features:

Dual-core ARM Cortex-M0+ processor

Supports MicroPython and C/C++

Multiple GPIO pins

Low power consumption

ARDUINO VS ESP32 VS RASPBERRY PI			
	 ARDUINO	ESP32	
Type	Microcontroller board	Wi-Fi/Bluetooth-enabled microcontroller	Single-board computer
Processing Power	8-bit / 16-bit, low MHz	32-bit dual-core, ~240 MHz	64-bit ARM CPU, multi-core GHz
Connectivity	None (basic UART, SPI, I ² C)	Wi-Fi, Bluetooth (BLE), Mesh, Matter (new chips)	Wi-Fi, Bluetooth, Ethernet, USB, HDMI, Camera
OS Support	None (bare-metal programming)	Optional RTOS (FreeRTOS)	Full OS (Linux, Raspberry Pi OS, Ubuntu, etc.)
Programming	Arduino IDE (C/C++)	Arduino IDE, MicroPython, ESP-IDF (C++)	Python, C, C++, Java, Node.js, etc.
Present Use Cases	Education, robotics basics, real-time control	IoT devices, smart home, sensors, wearables	AI, ML, computer vision, edge servers, robotics
Future Role	Remains in education & simple control	Core in IoT, smart healthcare, smart cities, Industry 4.0	AI +IoT gateway, robotics, cloud-edge integration

B. Basic Sensors

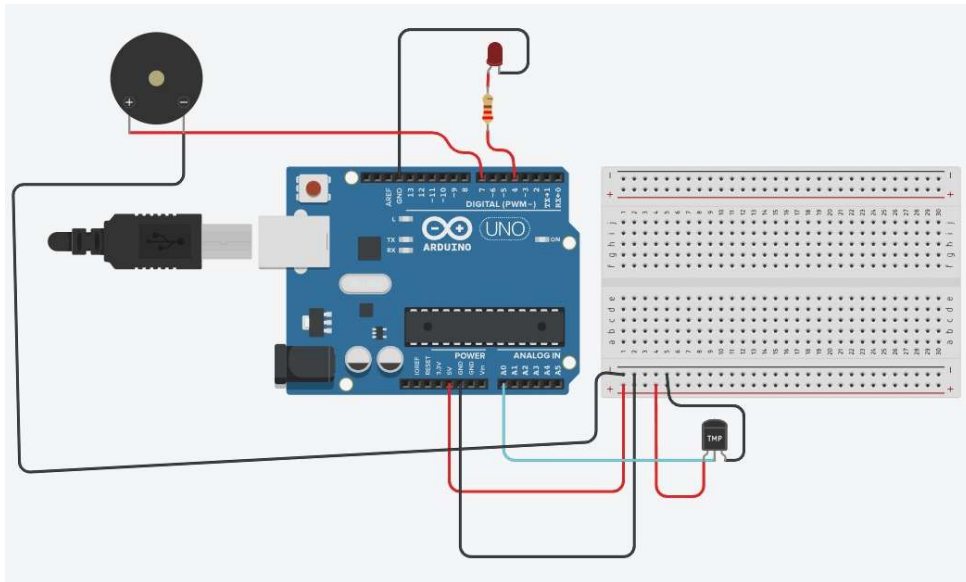
Sensors are electronic devices that detect physical changes in the environment and convert them into electrical signals that can be processed by a microcontroller.

Sensors used:

1. Temperature Sensor
2. Motion Sensor (PIR)
3. Light Sensor (LDR)
4. Ultrasonic Sensor
5. Humidity Sensor
6. LiDAR Sensor
7. Gyroscope Sensor
8. Accelerometer Sensor
9. Gas Sensor (MQ Series)

3. Mini Project:

Smart Temperature Monitor:



4. Project Explanation:

1. Project Overview:

The Smart Temperature Monitor is an embedded system project designed to monitor surrounding temperature using a TMP36 temperature sensor. The system provides different alert indications using an LED and a buzzer when the temperature crosses specific threshold values.

2. Components Used:

Component	Purpose
Arduino Uno	Main microcontroller used for processing
TMP36 Temperature Sensor	Measures temperature
LED	Visual alert indication

220Ω Resistor	Protects LED from excess current
Buzzer	Audio alert indication
Breadboard	Circuit connections
Jumper Wires	Electrical connections

3. Simulation Software Used:

The project is developed using an Arduino Uno and simulated using Tinkercad.

4. Working Principle:

- The TMP36 temperature sensor continuously measures the surrounding temperature and sends an analog voltage signal to the Arduino Uno.
- The Arduino reads this analog value through pin A0 and converts it into temperature in degree Celsius using the programmed formula.
- Based on the temperature value, the Arduino activates the LED and buzzer according to predefined conditions.
- LED indicates visual warning.
- Buzzer indicates audio warning
- The system continuously monitors temperature in real time.

5.Code:

```

1  const int sensorPin = A0;
2  const int ledPin = 4;
3  const int buzzerPin = 7;
4
5  void setup() {
6      pinMode(ledPin, OUTPUT);
7      pinMode(buzzerPin, OUTPUT);
8  }
9
10 void loop() {
11     int rawReading = analogRead(sensorPin);
12     float voltage = rawReading * (5.0 / 1023.0);
13     float temperatureC = (voltage - 0.5) * 100;
14
15     if (temperatureC < 0) {
16         digitalWrite(ledPin, HIGH);
17         noTone(buzzerPin);
18     }
19     else if (temperatureC >= 30 && temperatureC <= 40) {
20         digitalWrite(ledPin, LOW);
21         tone(buzzerPin, 1000);
22     }
23     else if (temperatureC > 40) {
24         digitalWrite(ledPin, HIGH);
25         tone(buzzerPin, 1000);
26     }
27     else {
28         digitalWrite(ledPin, LOW);
29         noTone(buzzerPin);
30     }
31
32     delay(500);
33 }

```

6. Condition-Based Operation:

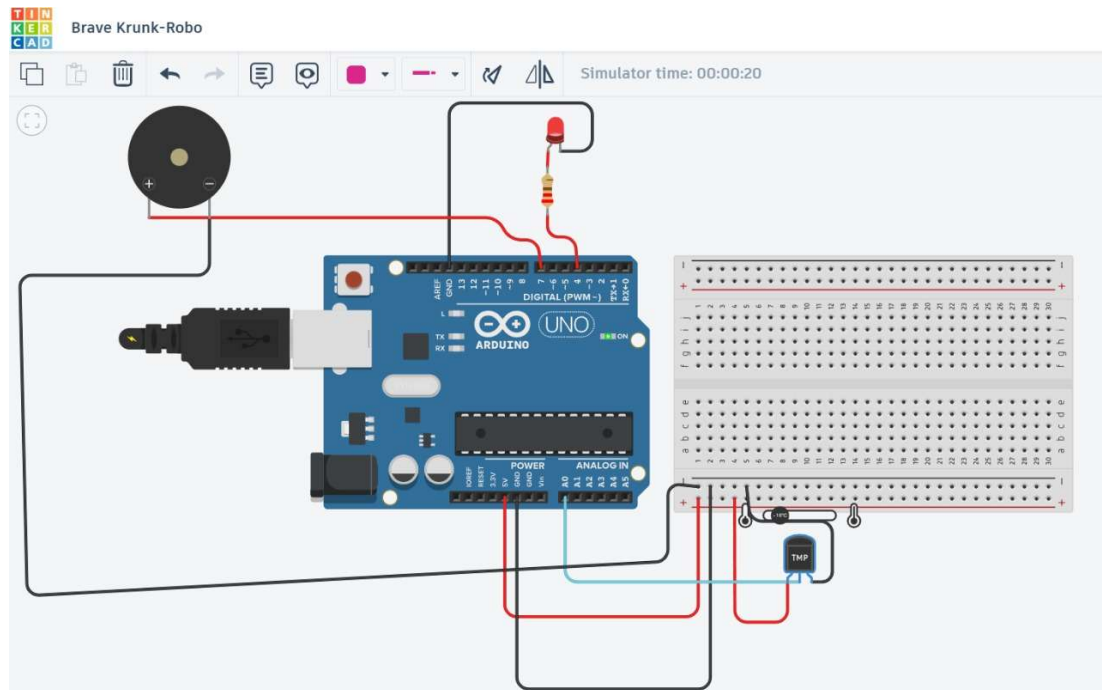
Condition 1 – Negative Temperature

Condition: Temperature less than 0°C

Output:

LED glows

Buzzer remains OFF



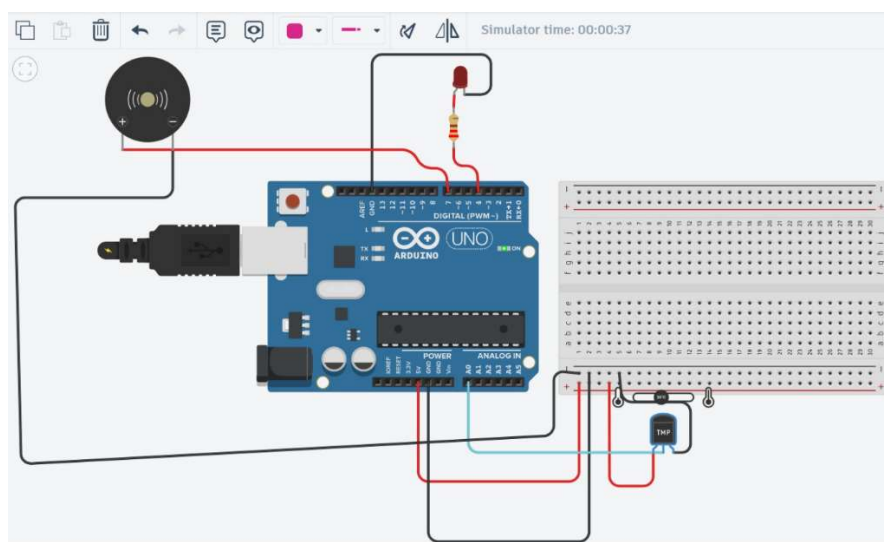
Condition 2 – Moderate High Temperature

Condition: Temperature between 30°C and 40°C

Output:

Buzzer turns ON

LED remains OFF



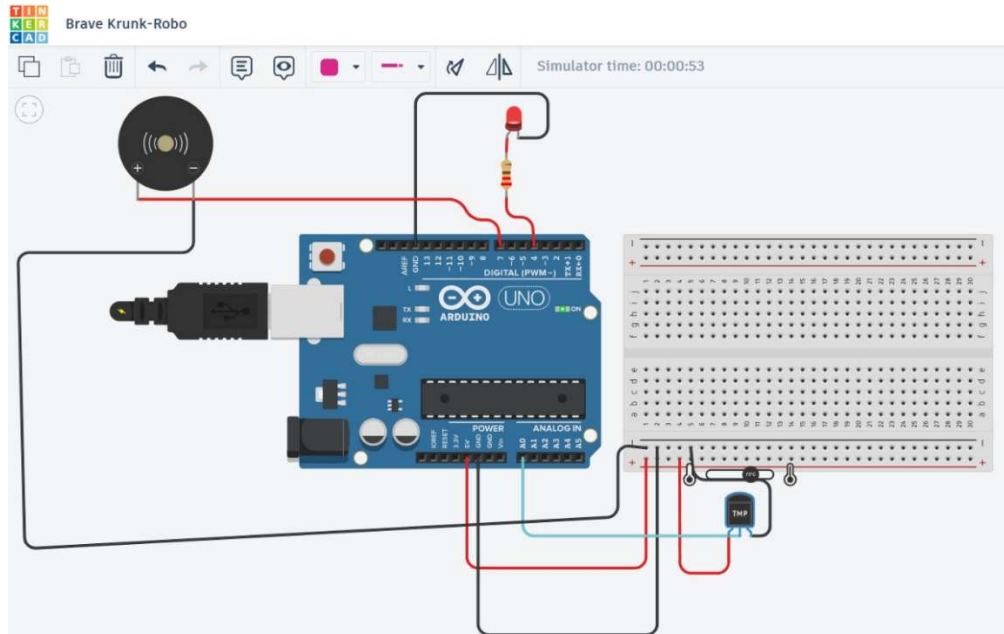
Condition 3 – Very High Temperature

Condition: Temperature greater than 40°C

Output:

LED glows

Buzzer turns ON



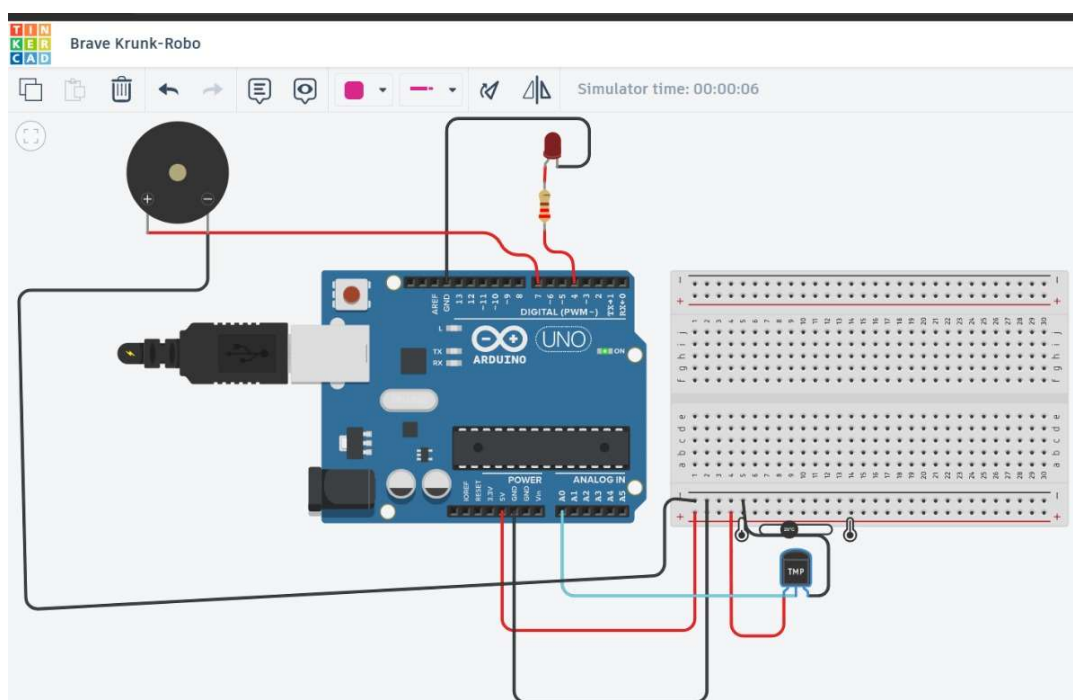
Condition 4 – Normal Temperature

Condition: Temperature between 0°C and 29°C

Output:

LED OFF

Buzzer OFF



7. Real-World Applications:

Case Study: Server Room Temperature Monitoring

1. A Smart Temperature Monitor can be used in server rooms where computers and networking devices generate large amounts of heat.
2. The TMP36 sensor continuously checks the room temperature and sends data to the Arduino Uno. If the temperature rises above 30°C, the buzzer gives a warning alert. If the temperature exceeds 40°C, both the buzzer and LED activate to indicate a dangerous overheating condition.
3. This helps technicians identify cooling problems early and prevents damage to expensive electronic equipment.

8. Conclusion:

The Smart Temperature Monitor successfully demonstrates the use of embedded systems for real-time environmental monitoring. Using Arduino Uno and TMP36 sensor, the system continuously checks temperature and provides alerts based on predefined conditions. The project is simple, cost-effective, and useful for safety and automation applications.